Reto 1: Despliegue Continuo Robusto con un Pipeline de CI/CD

Contexto: El equipo de desarrollo está trabajando en una aplicación que se actualiza frecuentemente. Para automatizar el proceso de entrega de nuevas versiones y asegurar la calidad del software, es necesario implementar un pipeline de Integración Continua y Entrega Continua (CI/CD). Este pipeline debe construir automáticamente la aplicación, ejecutar pruebas para verificar su correcto funcionamiento y desplegarla en un entorno de pruebas o producción ante cada cambio en el código fuente, minimizando la intervención manual y acelerando el ciclo de vida del software.

Objetivo: Comprender y configurar un pipeline de Integración Continua y Entrega Continua (CI/CD) automatizado para la construcción, prueba y despliegue de una aplicación.

Descripción: Los estudiantes deberán seleccionar y configurar una herramienta de CI/CD popular (como Jenkins, GitLab CI, GitHub Actions o CircleCI), explorando sus conceptos fundamentales como stages, jobs y artifacts. El pipeline que definan debe activarse al detectar commits en una rama específica de un repositorio de código gestionado con Git. Las etapas mínimas requeridas son: clonar el repositorio, construir una imagen Docker de la aplicación utilizando un Dockerfile, ejecutar pruebas unitarias y/o de integración para asegurar la calidad del código antes del despliegue, etiquetar y subir la imagen Docker construida a un registro de contenedores accesible (como Docker Hub o un registro privado), y finalmente, desplegar la nueva versión de la aplicación en un entorno de destino (que puede ser local simulado con Docker Compose o un clúster de Kubernetes de desarrollo). Se espera que implementen un sistema básico de notificaciones (por correo electrónico o una plataforma de mensajería) para informar sobre el éxito o fallo de cada etapa del pipeline y que exploren mecanismos para gestionar de forma segura cualquier secreto necesario para el proceso (como credenciales de registro o claves de despliegue) utilizando las funcionalidades de la herramienta de CI/CD.

Consideraciones: Proporcionar ejemplos de archivos de configuración del pipeline (.gitlab-ci.yml, Jenkinsfile, .github/workflows/main.yml) para las herramientas sugeridas, asegurar que los estudiantes tengan acceso a cuentas gratuitas o de prueba en las herramientas de CI/CD y a un registro de contenedores, ofrecer un repositorio de código de ejemplo sencillo con una aplicación básica y algunas pruebas unitarias, fomentar una discusión sobre diferentes estrategias de branching en Git (Gitflow, GitHub Flow) y cómo se integran con el pipeline de CI/CD, y animar a la exploración de conceptos como la creación y gestión de artefactos del pipeline.

Referencias:

1. Jenkins Documentation
2. GitLab CI/CD Documentation
3. GitHub Actions Documentation

Reto 2: Monitorización y Observabilidad Integral con Prometheus, Grafana y Alertmanager

Contexto: Una vez que la aplicación está en funcionamiento en un entorno dinámico como Kubernetes, es crucial tener una visión detallada de su rendimiento y salud a nivel de infraestructura y aplicación. Esto permite identificar y diagnosticar problemas rápidamente, optimizar el uso de recursos, entender el comportamiento del sistema en tiempo real y tomar decisiones informadas sobre su escalabilidad y rendimiento futuro. Una solución de monitorización y observabilidad robusta es esencial para garantizar la fiabilidad y la calidad del servicio.

Objetivo: Implementar una solución completa de monitorización y observabilidad para una aplicación desplegada en Kubernetes, incluyendo la recolección de métricas del sistema (CPU, memoria, red) y de la aplicación (latencia, errores, peticiones), la visualización de estas métricas en dashboards intuitivos y personalizables, y la configuración de alertas proactivas basadas en umbrales definidos para ser notificados ante posibles problemas antes de que impacten a los usuarios.

Descripción: Los estudiantes deberán desplegar Prometheus en su clúster de Kubernetes utilizando Helm o archivos YAML, configurando la detección de servicios para descubrir automáticamente los objetivos a monitorizar (nodos, pods, contenedores). Además, instrumentarán su aplicación para exponer métricas personalizadas relevantes para su funcionamiento (como latencia de peticiones, tasas de error HTTP, tamaño de colas de procesamiento, uso de recursos específicos de la aplicación, etc.) utilizando una biblioteca cliente de Prometheus para el lenguaje de programación de la aplicación. Luego, desplegarán Grafana y lo conectarán a Prometheus como fuente de datos para crear dashboards visualmente informativos y personalizables que muestren las métricas clave de la infraestructura y de la aplicación, utilizando diferentes tipos de gráficos y paneles. Finalmente, implementarán Alertmanager para definir reglas de alerta basadas en las métricas recolectadas en Prometheus y configurar mecanismos de notificación (como correo electrónico, Slack o un webhook) para ser alertados cuando se detecten condiciones anómalas o problemas potenciales que requieran intervención.

Consideraciones: Proporcionar documentación detallada sobre la instalación y configuración de Prometheus, Grafana y Alertmanager en Kubernetes, ofrecer ejemplos de métricas personalizadas que podrían ser útiles para diferentes tipos de aplicaciones y cómo exponerlas, sugerir plantillas de dashboards de Grafana como punto de partida que muestren métricas comunes de Kubernetes y de aplicaciones, proporcionar ejemplos de reglas de alerta comunes y explicar cómo configurarlas en Prometheus y Alertmanager, y animar a los estudiantes a simular escenarios de problemas en su aplicación (aumento de carga, errores) para verificar que las alertas se activen correctamente y se reciban las notificaciones configuradas, explorando la configuración de silenciado y agrupamiento de alertas en Alertmanager.

Referencias:

1. Prometheus Documentation
2. Grafana Documentation
3. Alertmanager Documentation

Reto 3: Estrategias de Despliegue Avanzado en Kubernetes

Contexto: Al liberar nuevas versiones de una aplicación en producción en un entorno de Kubernetes, es fundamental minimizar cualquier interrupción del servicio para los usuarios y reducir el riesgo de introducir regresiones o errores que puedan afectar la estabilidad del sistema. Implementar estrategias de despliegue avanzadas permite una transición gradual y controlada a la nueva versión, facilitando la detección temprana de problemas en un subconjunto de usuarios o instancias antes de una liberación completa y proporcionando mecanismos para realizar rollbacks de forma segura y rápida en caso de fallos.

Objetivo: Aprender e implementar diferentes estrategias de despliegue en Kubernetes (Rolling Updates, Canary Deployments, y opcionalmente Blue/Green Deployments) para actualizar aplicaciones sin interrupciones significativas y con la capacidad de mitigar riesgos, comprendiendo las ventajas y desventajas de cada estrategia en diferentes escenarios de despliegue.

Descripción: Los estudiantes comenzarán con una aplicación desplegada en Kubernetes utilizando un Deployment. Luego, modificarán la configuración del Deployment para realizar una actualización utilizando la estrategia de RollingUpdate, observando cómo Kubernetes orquesta la creación gradual de nuevos pods con la nueva versión y la eliminación de los pods de la versión anterior, asegurando la disponibilidad continua de la aplicación. A continuación, implementarán una estrategia de Canary Deployment, desplegando una pequeña porción de la nueva versión (un número reducido de réplicas) junto con la versión estable para un subconjunto limitado de usuarios o tráfico, permitiendo la monitorización del rendimiento y la detección de errores en la nueva versión con un impacto reducido antes de una liberación completa. Opcionalmente, pueden explorar la estrategia Blue/Green, que implica desplegar la nueva versión en un conjunto separado de pods (el entorno "verde") mientras la versión actual sigue sirviendo tráfico en el conjunto de pods existente (el entorno "azul"), y luego redirigir todo el tráfico al entorno "verde" una vez que se considere estable. Se espera que los estudiantes observen el comportamiento de los pods durante las actualizaciones utilizando kubectl get pods -w, verifiquen la disponibilidad de la aplicación realizando peticiones durante el proceso y practiquen la realización de rollbacks utilizando kubectl rollout undo en caso de que se detecten problemas con la nueva versión.

Consideraciones: Proporcionar ejemplos de archivos YAML para configurar las diferentes estrategias de despliegue (strategy.type y strategy.rollingUpdate en el Deployment), sugerir herramientas de línea de comandos como kubectl get deployments, kubectl describe deployment y kubectl rollout status para observar el progreso y el estado de las actualizaciones, animar a los estudiantes a utilizar herramientas de simulación de tráfico para verificar la disponibilidad y el comportamiento de las diferentes versiones de la aplicación durante las actualizaciones canarias, y plantear escenarios específicos de fallos en la nueva versión para que practiquen cómo realizar rollbacks de manera efectiva y rápida, comprendiendo el historial de los rollouts con kubectl rollout history.

Referencias:

1. Kubernetes Deployments Documentation
2. Kubernetes Rolling Updates Documentation

3. Canary Deployments Explained

Reto 4: Enrutamiento Inteligente con Service Mesh (Linkerd o Istio)

Contexto: En arquitecturas de microservicios complejas desplegadas en Kubernetes, gestionar el tráfico entre los diferentes servicios, implementar políticas de ruteo sofisticadas para realizar pruebas A/B o despliegues canary a nivel de servicio, y asegurar la comunicación con características como la observabilidad y la seguridad (TLS mutuo) se vuelve un desafío inherente. Un Service Mesh proporciona una capa de infraestructura dedicada para manejar estas tareas de manera transparente para las aplicaciones, desacoplando la lógica de red de la lógica de negocio. **Objetivo:** Comprender y configurar el enrutamiento basado en cabeceras HTTP utilizando un Service Mesh (Linkerd o Istio) para controlar el flujo de tráfico entre diferentes versiones de una aplicación sin necesidad de modificar el código de la aplicación, permitiendo la implementación de estrategias de lanzamiento avanzadas y la experimentación con nuevas funcionalidades en producción con un riesgo controlado.

Descripción: Con una aplicación previamente desplegada en un clúster de Kubernetes donde un Service Mesh (Linkerd o Istio) esté instalado y configurado, los estudiantes deberán definir reglas de enrutamiento a través de las CRDs (Custom Resource Definitions) proporcionadas por el Service Mesh. El objetivo específico es configurar el Service Mesh para que, basándose en la presencia o el valor de una cabecera HTTP particular (por ejemplo, X-Feature-Flag: new-version o User-Agent: experimental), el tráfico se dirija a una versión específica de la aplicación (un conjunto diferente de pods). Todas las demás peticiones que no contengan o no cumplan con el valor de la cabecera deberán ser enrutadas a la versión principal o estable de la aplicación. Se espera que los estudiantes demuestren y verifiquen el correcto funcionamiento de estas reglas de enrutamiento enviando peticiones HTTP a la aplicación utilizando herramientas como curl o Postman, modificando las cabeceras de las peticiones y observando cómo el tráfico se distribuye a las diferentes versiones de la aplicación en función de las reglas configuradas en el Service Mesh.

Consideraciones: Proporcionar documentación detallada y ejemplos de configuración específicos para el Service Mesh elegido (Linkerd o Istio) sobre cómo implementar el enrutamiento basado en cabeceras (por ejemplo, utilizando TrafficSplit y ServiceProfile en Linkerd o VirtualService y DestinationRule en Istio), sugerir herramientas de línea de comandos como curl o herramientas GUI como Postman para construir y enviar peticiones HTTP personalizadas con diferentes cabeceras, animar a los estudiantes a explorar las herramientas de observabilidad que proporciona el Service Mesh (como dashboards de tráfico, métricas de latencia y errores) para visualizar el flujo del tráfico bajo las reglas configuradas, y discutir los beneficios de utilizar un Service Mesh para la gestión del tráfico en comparación con soluciones a nivel de Ingress o balanceadores de carga tradicionales.

Referencias:

1. Linkerd Documentation - Traffic Splitting
2. Istio Documentation - Traffic Management
3. Service Mesh Interface (SMI) Specification

Reto 5: Definiendo tu Infraestructura en la Nube con Terraform (o Pulumi)

Contexto: La gestión manual de la infraestructura en la nube a través de consolas web o interfaces de línea de comandos puede ser propensa a errores humanos, inconsistente entre entornos y difícil de auditar y versionar. La Infraestructura como Código (IaC) ofrece una solución al permitir definir y gestionar la infraestructura utilizando archivos de configuración declarativos versionados, lo que facilita la automatización del aprovisionamiento, la repetibilidad de los despliegues, el control de cambios a través de sistemas de control de versiones y la colaboración entre equipos.

Objetivo: Aprender a definir y aprovisionar recursos fundamentales en un proveedor cloud utilizando una herramienta de Infraestructura como Código (Terraform o Pulumi), comprendiendo el ciclo de vida de la infraestructura gestionada con IaC (planificación, creación, modificación y destrucción) y los beneficios que aporta en términos de consistencia y automatización.

Descripción: Los estudiantes deberán seleccionar una herramienta de IaC (Terraform o Pulumi) y un proveedor de nube (AWS, Azure, GCP u otro que ofrezca una cuenta gratuita o de prueba). Utilizando la sintaxis específica de la herramienta elegida, definirán un conjunto básico de recursos de infraestructura necesarios para alojar una aplicación simple y accesible. Este conjunto incluirá al menos un grupo de recursos (o equivalente para la organización), una red virtual (VPC, VNet) con un rango de direcciones IP específico y una subred dentro de esa red, una instancia de servidor virtual (EC2, VM, Compute Engine instance) con un sistema operativo Linux básico ubicada en la subred, y una regla de seguridad (Security Group, NSG, Firewall Rule) que permita el acceso SSH a la instancia para la administración. El reto implica utilizar la herramienta de IaC para planificar los cambios que se realizarán en la infraestructura (terraform plan o pulumi preview), crear la infraestructura en el proveedor cloud (terraform apply o pulumi up), verificar la creación y configuración de los recursos a través de la consola del proveedor cloud, y finalmente, utilizar la misma herramienta para destruir todos los recursos aprovisionados (terraform destroy o pulumi destroy), asegurando la limpieza del entorno y la prevención de costos innecesarios. Opcionalmente, se puede incluir una tarea de modificación de la infraestructura (como cambiar el tipo de instancia, añadir una etiqueta o crear una regla de seguridad adicional) para practicar la gestión de cambios con IaC.

Consideraciones: Proporcionar guías paso a paso para la instalación y configuración de la herramienta de IaC y la configuración de las credenciales del proveedor cloud de forma segura (explorando el uso de variables de entorno o archivos de configuración específicos), ofrecer ejemplos básicos de archivos de configuración de IaC para los recursos solicitados en la sintaxis de Terraform o Pulumi, recordar enfáticamente la importancia de comprender y planificar los cambios antes de aplicarlos y de destruir los recursos al finalizar el ejercicio para evitar incurrir en costos inesperados, y opcionalmente, mostrar cómo visualizar la infraestructura creada a partir de los archivos de configuración de IaC utilizando herramientas como terraform graph o las capacidades de visualización de Pulumi.

Referencias:

1. Terraform Documentation
2. Pulumi Documentation

3. AWS Documentation, Azure Documentation, GCP Documentation

Reto 6: Exponiendo tu Aplicación de Forma Inteligente con Ingress y TLS en Kubernetes

Contexto: Para que las aplicaciones desplegadas en un clúster de Kubernetes sean accesibles desde el exterior de la red del clúster de forma segura y utilizando nombres de dominio intuitivos, es necesario configurar un punto de entrada que gestione el tráfico entrante. El recurso Ingress en Kubernetes permite definir reglas de enrutamiento a nivel de HTTP y HTTPS, y la habilitación de TLS (HTTPS) es crucial para proteger la comunicación con los usuarios mediante el cifrado de los datos transmitidos.

Objetivo: Aprender a exponer múltiples servicios en Kubernetes externamente de forma segura utilizando un recurso Ingress con enrutamiento basado en nombres de host y cifrado TLS (HTTPS), comprendiendo cómo dirigir el tráfico a diferentes aplicaciones o servicios basándose en el nombre de dominio solicitado por el cliente a través de un único punto de entrada gestionado por un Ingress Controller.

Descripción: Los estudiantes deberán tener desplegadas al menos dos aplicaciones o servicios web diferentes en su clúster de Kubernetes, cada uno expuesto a través de un Service (generalmente de tipo ClusterIP). El reto consiste en definir un recurso Ingress que configure el enrutamiento del tráfico basado en los nombres de host solicitados por los clientes (por ejemplo, las peticiones a app1.ejemplo.com se dirigen al primer servicio y las peticiones a app2.ejemplo.com al segundo servicio). Además, deberán configurar el Ingress para utilizar TLS (HTTPS) para ambos nombres de host. Esto implica generar o obtener certificados TLS (ya sean autofirmados para pruebas o de una autoridad certificadora), almacenarlos de forma segura en un Secret de tipo kubernetes.io/tls en el mismo namespace que el Ingress, y referenciar este Secret en la definición del Ingress para habilitar el cifrado para los nombres de host configurados. Los estudiantes también deberán configurar un Ingress Controller (como Nginx Ingress Controller o Traefik) en su clúster para que el recurso Ingress sea funcional. Finalmente, verificarán el acceso seguro a las aplicaciones a través de los nombres de host configurados utilizando HTTPS.

Consideraciones: Explicar la diferencia entre HTTP e HTTPS y la importancia del cifrado TLS, detallar los pasos para generar o adquirir certificados TLS y cómo crear el Secret en Kubernetes, proporcionar ejemplos de configuraciones de Ingress con enrutamiento basado en host y la sección tls, guiar a los estudiantes en la instalación y configuración de un Ingress Controller popular, y sugerir herramientas para verificar los certificados TLS de los sitios web expuestos.

Referencias:

1. Kubernetes Ingress Documentation
2. Traefik Documentation
3. Cert-manager Documentation (para la gestión automatizada de certificados)

Reto 7: Automatización de la Configuración con Ansible

Contexto: La gestión manual de la configuración de servidores y aplicaciones puede ser tediosa y propensa a errores, especialmente en entornos con muchos sistemas. Las herramientas de automatización de la configuración, como Ansible, permiten gestionar la configuración de forma centralizada, repetible y eficiente.

Objetivo: Aprender a utilizar Ansible para automatizar tareas de configuración en múltiples servidores, comprendiendo los conceptos de inventarios, playbooks y módulos.

Descripción: Los estudiantes deberán configurar un entorno de Ansible (lo que puede implicar instalar Ansible en una máquina de control y configurar el acceso SSH a los servidores objetivo). Luego, crearán un inventario de servidores que Ansible gestionará. El reto principal consiste en escribir playbooks de Ansible para realizar tareas comunes de configuración, como instalar paquetes, configurar servicios, crear usuarios y gestionar archivos. Se espera que los estudiantes prueben sus playbooks en los servidores configurados en el inventario y verifiquen que la configuración se aplica correctamente. **Consideraciones:** Proporcionar una guía de instalación y configuración de Ansible, ofrecer ejemplos de playbooks para tareas comunes, explicar cómo gestionar las credenciales de acceso a los servidores de forma segura, y sugerir escenarios para probar la idempotencia de los playbooks (ejecutar el mismo playbook varias veces y verificar que el resultado es el mismo).

Referencias:

1. Ansible Documentation
2. Ansible Galaxy
3. Getting Started with Ansible

Reto 8: Gestión de Logs Centralizada con Elasticsearch, Fluentd y Kibana (EFK)

Contexto: En un entorno distribuido con múltiples aplicaciones y servidores, la gestión y el análisis de logs se vuelve complejo. Una solución de gestión de logs centralizada permite recolectar, procesar, almacenar y visualizar logs de diferentes fuentes en un único lugar, facilitando la identificación y resolución de problemas, la monitorización del comportamiento del sistema y el análisis de tendencias a lo largo del tiempo.

Objetivo: Implementar una solución de gestión de logs centralizada utilizando Elasticsearch para el almacenamiento y búsqueda, Fluentd para la recolección y procesamiento, y Kibana para la visualización y el análisis de logs.

Descripción: Los estudiantes deberán configurar Fluentd (o Fluent Bit como alternativa ligera) en uno o más servidores o dentro de un clúster de Kubernetes para recolectar logs de diferentes fuentes (por ejemplo, logs de aplicaciones, logs del sistema operativo). Luego, configurarán Fluentd para procesar y enviar estos logs a un clúster de Elasticsearch. Finalmente, desplegarán Kibana y lo conectarán a Elasticsearch para crear dashboards y visualizaciones que permitan buscar, filtrar y analizar los logs de manera eficiente. Se espera que los estudiantes exploren diferentes patrones de logs, aprendan a configurar Fluentd para parsear logs estructurados y no estructurados, y creen visualizaciones útiles en Kibana para la monitorización y el troubleshooting de aplicaciones.

Consideraciones: Proporcionar guías detalladas para la instalación y configuración de Elasticsearch, Fluentd y Kibana, ofrecer ejemplos de configuraciones de Fluentd para diferentes tipos de logs (por ejemplo, logs de aplicaciones en JSON, logs de acceso de servidores web), explicar cómo configurar índices y mappings en Elasticsearch para optimizar el almacenamiento y la búsqueda de logs, y sugerir tipos de visualizaciones útiles en Kibana (gráficos de líneas para tendencias, tablas para logs sin procesar, mapas para logs geolocalizados si aplica).

Referencias:

1. Elasticsearch Documentation
2. Fluentd Documentation
3. Kibana Documentation